

Phase 00 — Linux & Terminal Basics (Prerequisites)

The absolute foundation. Every later phase assumes you can do everything in this chapter without thinking. (Phase 4 covers server administration — users, systemd, SSH, firewall. This phase is about being fluent in the terminal itself.)

1. What is Linux, really?

- **Kernel** — the core program managing CPU, memory, disks, network. Written by Linus Torvalds, 1991.
- **Distribution (distro)** — kernel + tools + package manager: Ubuntu, Debian, Fedora, Alpine. We use **Ubuntu** (and Ubuntu Server on VPS/cloud) — largest community, most tutorials, default on most clouds.
- **Shell** — the program that reads your commands: `bash` (default on servers), `zsh` (your machine). Same fundamentals.
- **Terminal** — the window the shell runs in.

Analogy: kernel = engine, distro = the whole car built around it, shell = the steering wheel, terminal = the driver's seat.

2. The command line — anatomy

```
ls  -l  -a  /var/log
└┐  └┐  └┐
command flags argument
```

- Flags customize behavior; combine them: `ls -la`.
- `man ls` = the manual for any command (q to quit). `ls --help` = quick version.
- **Tab completion** — type `cd /va` + Tab. Use it constantly; it prevents typos.
- **History** — ↑/↓ arrows; `history`; `Ctrl+R` = search past commands (life-changing).
- `clear` or `Ctrl+L` clears the screen.

3. Moving around & handling files (muscle memory required)

```
pwd                # where am I
ls; ls -lah        # list; -l long, -a hidden (.files), -h human sizes
cd /var/log        # absolute path (starts with /)
cd ..; cd -; cd ~; cd # up one; back to previous; home; home
mkdir -p projects/api # create (-p: parents too)
touch notes.txt    # create empty file
cp a.txt b.txt     # copy; cp -r dir1 dir2 for folders
mv a.txt docs/     # move OR rename (mv old.txt new.txt)
rm a.txt           # delete (NO trash bin — gone is gone)
rm -rf dir/        # delete folder recursively — triple-check the path
```

Path rules: `/` root, `~` home (`/home/raju`), `.` here, `..` parent. Absolute paths start with `/`; everything else is relative to `pwd`.

4. Reading & searching files

```
cat file.txt           # print whole file
less file.txt          # page through (Space next page, / search, q quit)
head -20 f; tail -20 f # first/last lines
tail -f app.log        # ★ follow live as lines are appended – logs tool #1
grep "ERROR" app.log   # find lines containing text
grep -i "error" app.log # case-insensitive
grep -rn "TODO" src/   # recursive through a folder, with line numbers
wc -l file.txt         # count lines
find . -name "*.java"  # find files by name
```

5. Pipes and redirection — the superpower

The `|` pipe sends one command's output into the next. Small tools compose into powerful one-liners:

```
history | grep ssh      # what ssh commands did I run?
cat access.log | grep " 500 " | wc -l  # how many 500 errors?
ps aux | grep java      # find java processes
ls -lah | less          # page long output

command > file.txt       # write output to file (overwrite)
command >> file.txt      # append
command 2> errors.txt    # redirect errors (stderr)
command > all.txt 2>&1    # both output and errors
```

Analogy: pipes = a kitchen assembly line — chop | fry | plate. Each station does one thing well.

6. Installing software — apt

Ubuntu's package manager (like npm/composer for the OS):

```
sudo apt update        # refresh the package catalog (do this first)
sudo apt install curl git httpd # install
sudo apt upgrade       # update installed packages
apt search postgres    # find packages
sudo apt remove pkg    # uninstall
```

`sudo` = run as administrator (root). It will ask your password. Rule: use sudo only when needed (installing, editing system files) — never for daily work.

7. Editing files in the terminal — nano (and surviving vim)

```
nano file.txt          # friendly editor: Ctrl+O save, Ctrl+X exit, Ctrl+W search
```

You WILL land in **vim** by accident someday (`git commit` etc.): press `i` to type, `Esc` then `:wq` Enter to save+quit, `:q!` to quit without saving. That's enough to survive.

8. Processes & system — quick look (deep dive in Phase 4)

```
ps aux          # running processes
top / htop      # live CPU/memory view (q quits)
kill <PID>      # stop a process
df -h           # disk space
free -h         # memory
uname -a        # kernel/OS info
which java      # where is a command installed
echo $PATH      # folders searched for commands
```

9. Environment variables

Named values every program can read — how config reaches apps in production (Docker/K8s use them heavily):

```
echo $HOME; echo $USER; env          # see them
export API_KEY=abc123                 # set for this shell session
echo $API_KEY
# permanent: add the export line to ~/.bashrc (or ~/.zshrc), then: source ~/.zshrc
```

10. Keyboard shortcuts that make you fast

Tab	complete file/command	Ctrl+R	search history
Ctrl+C	kill current command	Ctrl+L	clear screen
Ctrl+A / Ctrl+E	start/end of line	Ctrl+W	delete last word
Ctrl+D	exit shell / end input	!!	repeat last command (sudo !!)

11. Common beginner mistakes

- Fearing the terminal and reaching for GUI tools — fluency only comes from daily use.
- `rm -rf` with wrong path (there is no undo). Type the path, then look at it, then Enter.
- Running everything with `sudo`.
- Spaces in filenames without quotes: `cat "my file.txt"`.
- Ignoring case sensitivity: `File.txt` $\neq$ `file.txt` on Linux.
- Copy-pasting commands from the Internet without understanding them (especially with `sudo`!).

12. Interview questions

1. Absolute vs relative path? What do `.`, `..`, `~` mean?
2. What does a pipe `|` do? Build a one-liner counting ERROR lines in a log.
3. `>` vs `>>` vs `2>`?
4. How do you watch a log file live?
5. What is `$PATH` and what happens when you type a command name?
6. What is an environment variable and why do production apps use them for config?

13. LAB — 30 minutes of muscle memory

```
# 1. build & explore a small tree
mkdir -p ~/lab/{src,logs,config} && cd ~/lab
echo "server.port=8080" > config/app.properties
for i in 1 2 3 4 5; do echo "$(date) INFO request $i ok" >> logs/app.log; done
echo "$(date) ERROR database timeout" >> logs/app.log
ls -lah; ls -R

# 2. read & search
cat logs/app.log; tail -3 logs/app.log
grep ERROR logs/app.log
grep -c INFO logs/app.log

# 3. live log following (two terminals)
tail -f logs/app.log                                # terminal 1
echo "$(date) ERROR disk full" >> ~/lab/logs/app.log  # terminal 2 → watch T1 update!

# 4. pipes
history | grep grep
ps aux | grep $USER | wc -l

# 5. edit
nano config/app.properties    # change the port, save, verify with cat

# 6. cleanup practice – CAREFULLY
cd ~; rm -rf ~/lab
```

14. ASSIGNMENT 00 (submit to Claude)

1. Do the lab; paste the output of steps 2 and 4.
2. Write one-liners for: (a) count lines containing " 404 " in `access.log` ; (b) show the last 50 lines of a log and keep following it; (c) find every `.properties` file under `~/projects` ; (d) save the list of running processes to `procs.txt` including errors.
3. Explain, as to a junior: what happens step-by-step when you type `htop` and press Enter (hint: `$PATH`), and what `command not found` means.
4. What's the difference between `rm file` , `rm -r dir` , `rm -rf dir` ? When is `-f` dangerous?
5. Set an environment variable `APP_ENV=dev` permanently for your shell, prove it survives a new terminal, and explain which file you used and why.